

HTML and Semantic Structure

Landmarks, heading hierarchy, lists, figures and meaningful link text: turning content into a document that a browser, a search tool and a screen reader can all understand.



Learning outcomes

By the end of this session you can build, read and audit a semantic document.

Structure

Write a page as one h1, nested h2/h3 sections and correct landmarks — before any CSS is applied.

Choose

Pick the element a purpose actually needs: link vs button, list vs table, figure vs bare image.

Verify

Read a page's accessibility tree and heading outline in browser DevTools and judge whether it holds together.

From page appearance to page meaning

A visually polished page can still have a broken document outline.

CSS can make almost any element look like a heading, a button or a list item. None of that visual styling changes what a browser, a search index or assistive technology believes the element actually is.

What CSS changes

Size, colour, spacing, position on screen — the presentation layer a sighted mouse user reads first.

What HTML changes

The role each part plays: is this a heading, a link, a list, a data table, a form control?

Week 2's question: what is this content, and how does each part relate to the whole document?

HTML is a vocabulary, not a style sheet

Tags describe the role of content; they do not describe its appearance.

A div and a button can be styled identically, but only one of them is a control that keyboard users, screen readers and browser autofill already know how to operate.

div / span

Generic containers with no inherent role. Use them only when no more specific element fits.

button, a, table, ul

Elements with a built-in role, keyboard behaviour and accessibility-tree mapping already defined by the HTML spec.

The rule

Reach for the specific element first. Recreate its behaviour in a div only if nothing native fits the case.

The minimum meaningful document

Three pieces of metadata a browser needs before it can render or announce anything.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Garden guide</title>
  </head>
  <body>
    <main>...</main>
  </body>
</html>
```

doctype fixes standards mode; lang sets a starting pronunciation and translation language; title names the page in tabs, bookmarks and search results.

Language metadata

The lang attribute is invisible on screen and essential off screen.

For assistive technology

A screen reader uses lang to choose a pronunciation engine — without it, English text can be read with the wrong accent rules, or vice versa.

For everyone else

Browsers use lang to offer translation and to select hyphenation and quotation-mark rules for the correct language.

Set lang on <html> for the whole page, and again on any element whose content changes language, e.g. lang="fr" on a quoted French phrase.

The page outline

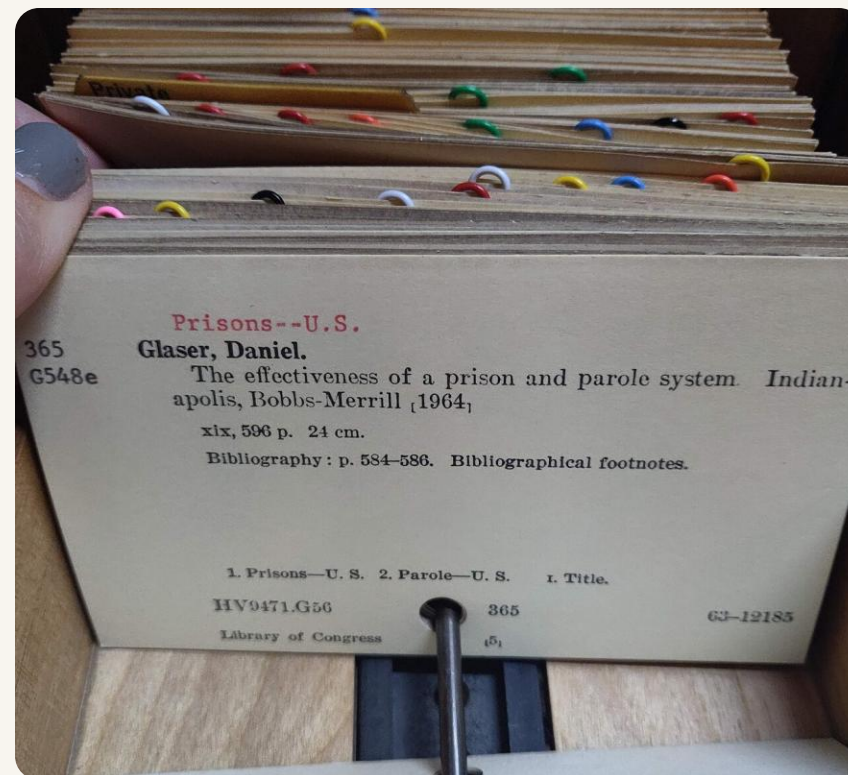
A heading sequence is an argument about what matters most.

Strip away every paragraph and read only the headings, top to bottom. If that sequence does not tell a coherent story on its own, the outline needs work before the prose does.

A library catalog is the same idea

Cards are filed by heading and subject so a visitor can navigate the collection without reading every card — exactly what a screen reader's heading list does for a page.

Screen readers can list all headings on a page instantly, letting a visitor jump straight to a section by name.



One h1, a disciplined h2/h3

Nesting rules that keep the outline navigable.

h1

one per page — the page's central subject or task

h2

each major section of that page

h3

only nested inside the h2 section it belongs to

Never nest a heading level for size alone — skip a level and a screen-reader outline will read as a mistake, not a design choice.

Heading audit exercise

A four-step method to check any page's outline before you trust it.

1

Open the page and list every heading in document order, ignoring visual size.

2

Check there is exactly one h1 and it names the page's subject.

3

Check every h3 sits inside an h2 section, with no skipped levels.

4

Read the list alone: does it summarise the page without the paragraphs?

Landmarks

header, nav, main, aside and footer let readers move between meaningful regions.

header

introductory content or site branding

nav

a group of navigation links

main

the unique content of this page

aside

related but tangential content

footer

closing metadata for the page or section



A physical wayfinding sign names distinct destinations the same way landmarks name distinct regions of a page.

The main and nav landmarks

The two landmarks almost every page needs.

One main region

Wrap only the page's unique content in main. Repeated site chrome — the logo, primary nav, footer links — belongs outside it.

A nav landmark per group

Wrap a meaningful set of navigation links in nav. A page can have more than one — primary nav and a table-of-contents nav are both valid.

A screen-reader user can jump straight to main with a single keystroke, skipping repeated header and navigation content on every page.

Article and section

Two containers for related content, with different rules for reuse.

article

A self-contained item that would still make sense if syndicated elsewhere on its own: a blog post, a comment, a product card.

section

A thematic grouping of content that belongs in this page, typically introduced by its own heading.

Test: could this block be pulled out and read alone, with its own heading, in a different context? If yes, it is an article.

A semantic article

The article element in context.

```
<article>
  <h1>Weekend repair clinic</h1>
  <p>Bring a small household item and
    a ten-minute problem.</p>
  <p>Saturdays, 10am, community hall.</p>
</article>
```

This block keeps its own heading and could be lifted into a listing page or an RSS feed and still make sense on its own.

Lists carry structure

Choosing ul, ol or dl is a claim about the relationship between items.

ul

an unordered group — order does not carry meaning

ol

a sequence — order is the point, e.g. instructions

dl

term/definition pairs — a glossary or a key-value list

A styled div stack of items announces nothing about count or order to anyone not reading it visually.

Why a list matters

A screen reader announces list membership before a reader commits to exploring it.

Before reading the first item, a screen reader can announce “list, 5 items” — letting a visitor decide whether to explore the whole list or skip past it.

Without a real list

A visitor gets no count, no boundary and no way to skip past a long unordered block efficiently.



Links vs buttons

Different elements make different promises to a visitor.

a — a link

Navigates: changes the URL, moves to a new page or to an anchor within the current one. Always has an href.

button — a button

Performs an action on the current page: opens a menu, submits a form, adds an item — without changing location.

Styling a link to look like a button does not give it keyboard-activation-with-Space or the button accessibility role.

A link is not a button

The same visual shape, two different underlying contracts.

```
<a href="/events">Browse events</a>
```

```
<button type="button" onclick="openFilters()">  
  Open filters  
</button>
```

Keyboard behaviour differs too: Enter activates both, but Space also activates a button — never a link.

Images: four roles, one decision

Decide the role before writing alternative text.

Decorative

`alt=""` — announced as nothing, skipped entirely

Informative

`alt` describes the information the image conveys

Functional

`alt` describes the action the image performs, e.g. a link icon

Complex

`alt` is a short label; a full description sits nearby in text



Writing alternative text

The useful description depends on the surrounding purpose, not the pixels.

Weak

“Image of a person.” Describes nothing a reader could not already guess, and adds no information.

Useful

“Student testing a website with keyboard navigation on a laptop.”
Communicates what a sighted reader takes from the image in context.

If the surrounding caption already states the fact, the alt text can be short — it should never repeat the caption word for word.

Figures and captions

Pair an image or diagram with a visible, explicit explanation.

```
<figure>
  
  <figcaption>
    Figure 1. The bicycle-light repair sequence.
  </figcaption>
</figure>
```

figcaption is announced as connected to its figure — unlike a nearby paragraph, which a screen reader cannot associate automatically.

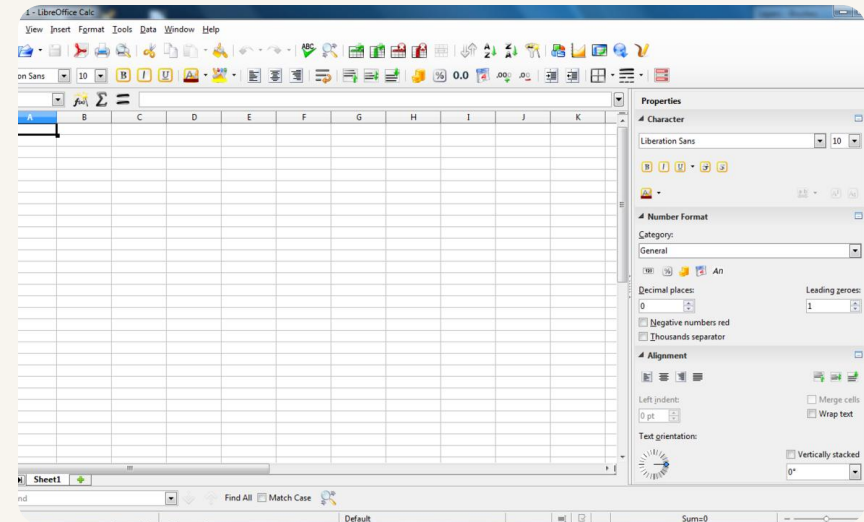
Tables for data, never for layout

A table claims a two-dimensional relationship between rows and columns.

Reach for table only when a reader genuinely needs to compare values across both a row and a column. Page layout belongs to CSS.

The tell

If you would describe the content as “rows and columns of related values,” it is data. If you are just arranging boxes on screen, it is not.



Header cells connect data to meaning

scope tells assistive technology which header applies to which cell.

```
<table>
  <thead>
    <tr><th scope="col">Week</th><th scope="col">Topic</th></tr>
  </thead>
  <tbody>
    <tr><td>2</td><td>Semantic HTML</td></tr>
  </tbody>
</table>
```

Without scope, a screen reader can only guess which header describes a given data cell in a large table.

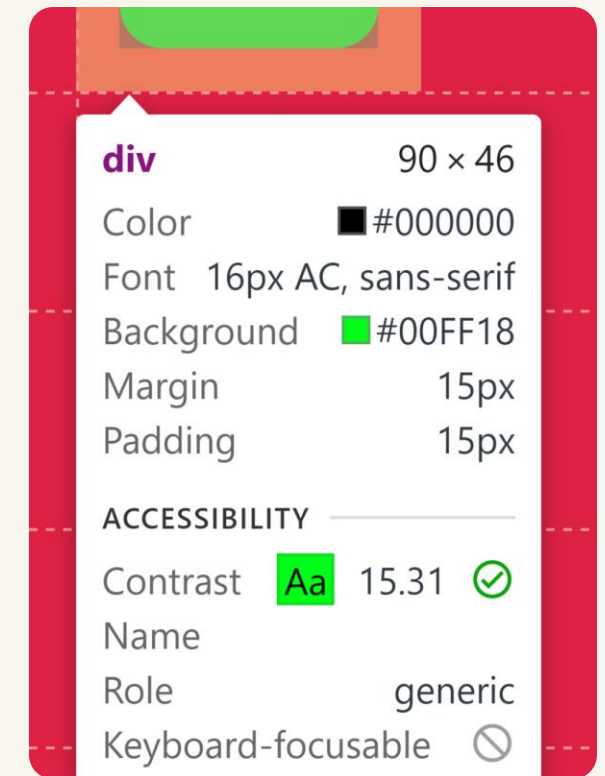
Forms need visible labels

Placeholder text disappears the moment a visitor starts typing.

A label stays on screen, is clickable to focus its control, and is what a screen reader announces before reading the field's current value.

Placeholder is not a label

Placeholder text vanishes on input, offers no persistent context, and often fails colour-contrast checks even while visible.



Labels, connections and errors

for/id ties a label to its control; aria-describedby ties an error to it too.

```
<label for="email">Email address</label>
<input id="email" type="email"
  aria-describedby="email-err">
<p id="email-err">Must include an @ symbol.</p>
```

Weak signal

A red border alone tells a screen-reader user nothing, and a colour-blind visitor little.

Connected message

Text describing the problem, linked with aria-describedby, announced when focus lands on the field.

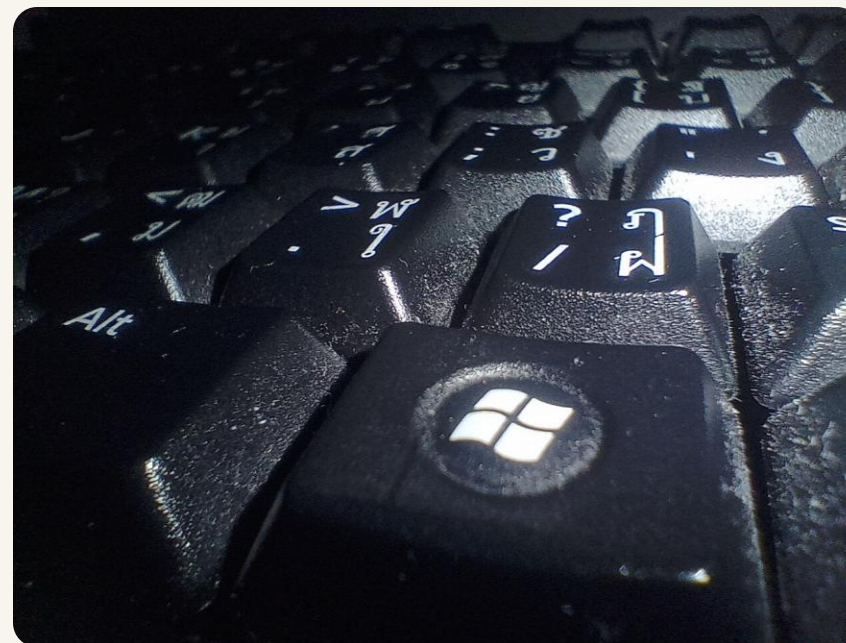
Keyboard access and testing

A page should work completely without a pointer.

**Press Tab through the whole page. Can you see where focus sits?
Can you reach every link, button and form field, in a sensible order?**

Source order sets tab order

CSS can rearrange the visual layout, but keyboard focus follows the HTML source order — keep that order logical.



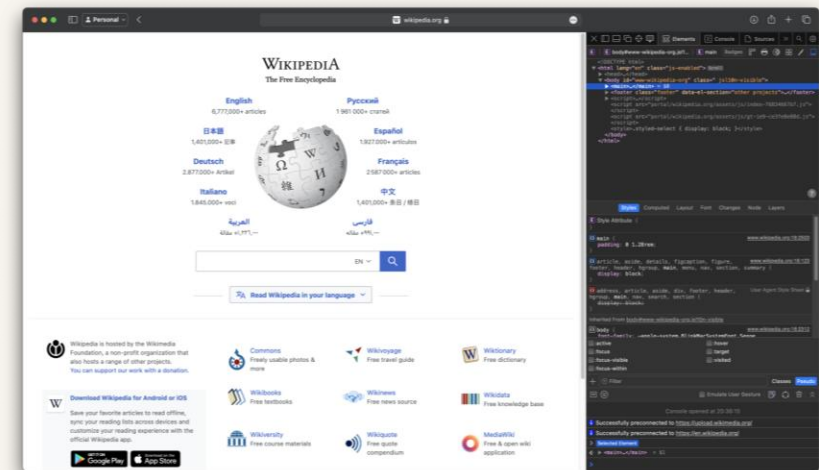
Browser inspection and the accessibility tree

DevTools shows the structure a screen reader actually receives.

The accessibility tree is a browser's parsed summary of every element's role, name and state — the same data a screen reader consumes.

What to check

Inspect the elements panel for headings and landmarks, then the accessibility tab for each control's computed role and accessible name.



Studio audit

Apply today's method to a page your group has not built.

1

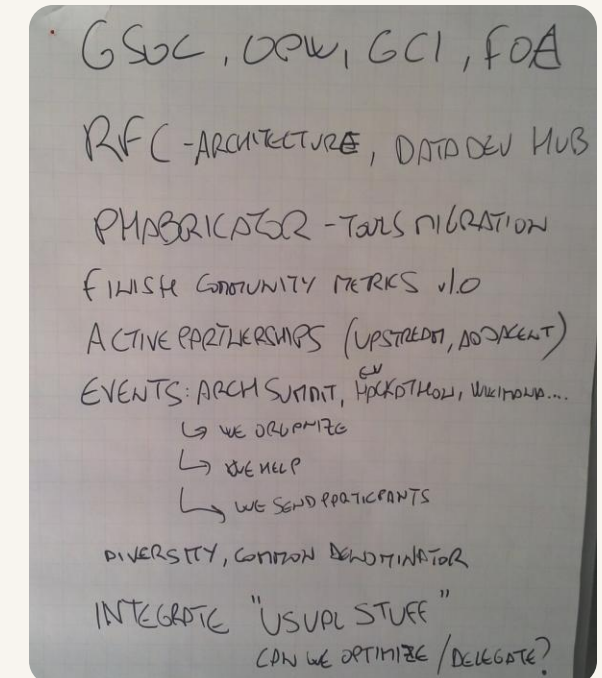
Choose a page. Map its headings and landmarks in document order.

2

Identify one element whose current HTML does not match its purpose.

3

Propose the correct element and explain what changes for a screen-reader user.



Studio build

Produce a small, genuinely semantic document.

1

Write one h1 naming the guide's subject.

2

Add three h2 sections, each with a clear, distinct purpose.

3

Include a nav list, one figure with a caption, and one small data table with header cells.

Evidence to submit: the HTML source, plus a one-line note on why each landmark and heading level was chosen.

Review checklist

What a reviewer checks before approving your structure.

Heading order

One h1; h2/h3 nest without skipping a level.

Landmarks

main holds unique content; nav groups real navigation.

Links and buttons

Links navigate; buttons act; none fakes the other.

Images and tables

Every image has a role; every table holds real data.

Labels

Every form control has a visible, connected label.

Recommended practice and next week

Continue the audit habit before Week 3.

Practice

MDN HTML learning area; W3C WAI tutorials on headings, landmarks and images; inspect three pages you use weekly.

Bring to Week 3

A page you find visually appealing, plus a short note on whether its structure would still make sense with the CSS removed.

Next week

Typography and visual hierarchy: how CSS strengthens a document that already makes sense.